

Python Programming Course

Table of Contents

Lecture 1: Introduction to Python Programming

Lecture 2: Working with Data Types and Variables

Lecture 3: Control Structures: Loops and Conditionals

Lecture 1: Introduction to Python Programming

Welcome to the Introduction to Python Programming lecture. In this session, we will explore the fundamental concepts of programming using Python, including understanding source code, syntax, the role of the Python interpreter, writing simple expressions, and how Python processes these commands.

1. Understanding Source Code

- **Source Code:** This is the sequence of instructions written in a programming language that a computer can execute. In Python, source code is written in plain text files with a .py extension.
- Example of a simple source code:

```
print('Hello, World!')
```

2. Syntax in Python

- **Syntax:** This refers to the set of rules that define the structure of statements in a programming language. Each programming language has its own syntax, and using incorrect syntax will result in errors.
- In Python, the syntax for printing a message is:

```
print('Your message here')
```

3. The Role of the Python Interpreter

- **Python Interpreter:** Python is an interpreted language, meaning that it directly executes the instructions written in the source code without the need for prior compilation. The interpreter translates the code into machine instructions that the computer can understand.
- The Python interpreter also provides an interactive environment where you can type commands and see immediate results. For example:

```
```
```

```
3 + 7 10 print('Hello') Hello ```
```

## 4. Writing Simple Expressions

- **Expressions:** An expression is a combination of values and operators that Python can evaluate to produce a result. For example,  $1 + 4 * 3$  is an expression that evaluates to 13 due to operator precedence.
- **Operators:** In Python, you can perform various arithmetic operations using operators:
  - + for addition
  - - for subtraction
  - \* for multiplication
  - / for division
  - % for modulus (remainder)
  - \*\* for exponentiation

## 5. Operator Precedence

- Python follows specific rules for the order of operations, known as precedence. For example, multiplication and division are performed before addition and subtraction. Parentheses can be used to alter this order.
- Example:
  - $1 + 3 * 4$  evaluates to 13.
  - $(1 + 3) * 4$  evaluates to 16.

## 6. Using the Python Console

- The Python console is where you can execute Python commands interactively. You can also write scripts in a text editor and run them in the console. For example:

```
age = input('How old are you? ') print('Your age is', age)
```

## 7. Variables and Data Types

- **Variables:** A variable is a named piece of memory that can store a value. You can assign a value to a variable using the assignment statement, e.g., `x = 5`.
- In Python, you can store different data types in variables, such as integers, floats, strings, and more.
- Example of variable assignment:

```
gpa = 3.14
```

## 8. Input and Output

- **Input:** You can read user input using the `input` function. The result can be stored in a variable for further processing.
- **Output:** The `print` function is used to display messages or results to the console.
- Example:

```
name = input('What is your name? ') print('Hello,', name)
```

## 9. Conclusion

In this lecture, we covered the basics of programming using Python. We discussed source code, syntax, the Python interpreter, writing simple expressions, and how Python processes commands. These concepts form the foundation for further learning in Python programming, and I encourage you to practice by writing your own Python code and experimenting with the interpreter. Thank you for your attention!

## Quiz Questions

**Q1: What file extension is used for Python source code?**

- .txt
- .java
- .py
- .html

*Correct: .py*

**Q2: What is the purpose of the Python interpreter?**

- To compile Python code into machine language
- To execute Python code directly
- To store Python files
- To edit Python source code

*Correct: To execute Python code directly*

**Q3: Which of the following is an arithmetic operator in Python?**

- &&
- ||

- +

- !

*Correct: +*

# Lecture 2: Working with Data Types and Variables

In this lecture, we will explore data types in Python, focusing on integers, floating-point numbers, and strings. We will learn how to create and manipulate variables, perform arithmetic operations, and utilize built-in functions for data handling.

## 1. Introduction to Data Types

Data types are the classification of data items. Python has several built-in data types that can be used to define variables. The most commonly used data types include: - Integers: Whole numbers, e.g., 5, -3, 42 - Floating-point numbers: Decimal numbers, e.g., 3.14, -0.001, 2.0 - Strings: A sequence of characters, e.g., 'hello', "Python programming"

## 2. Integers

Integers are whole numbers without a fractional component. In Python, you can perform various arithmetic operations using integers. Here are some examples: - Addition:  $5 + 3$  results in 8 - Subtraction:  $10 - 2$  results in 8 - Multiplication:  $4 * 2$  results in 8 - Division:  $16 / 2$  results in 8.0 (note that this produces a floating-point number) - Modulus:  $7 \% 3$  results in 1 (the remainder of the division)

## 3. Floating-Point Numbers

Floating-point numbers are used to represent real numbers that require a decimal point. In Python, you can perform the same arithmetic operations on floating-point numbers as you do with integers. Here are some examples: - Addition:  $2.5 + 1.5$  results in 4.0 - Subtraction:  $5.0 - 2.0$  results in 3.0 - Multiplication:  $3.0 * 2.0$  results in 6.0 - Division:  $5.0 / 2.0$  results in 2.5

## 4. Strings

Strings in Python are sequences of characters enclosed in single or double quotes. You can manipulate strings using various operations and methods. Here are some examples: - Concatenation: You can combine strings using the + operator. For example, "Hello" + "World" results in "Hello World". - Accessing Characters: You can access individual characters in a string using indexing. For example, in `name = 'Python'`, `name[0]` returns 'P'. - Slicing: You can extract a substring using slicing. For example, `name[1:4]` returns 'yth'. - String Length: Use the `len()` function to find the length of a string. For example, `len(name)` returns 6.

## 5. Variables

A variable is a named piece of memory that stores a value. In Python, variables are created using assignment statements. The syntax is as follows:

```
variable_name = value
```

For example: `x = 5` - `gpa = 3.14`

You can use variables in expressions. For example, if `x = 7`, then `x + 3` results in 10.

## 6. Performing Arithmetic Operations

You can perform arithmetic operations using variables. Here are some examples: - If `a = 10` and `b = 5`, then: - Addition: `result = a + b` results in 15 - Subtraction: `result = a - b` results in 5 - Multiplication: `result = a * b` results in 50 - Division: `result = a / b` results in 2.0 - Modulus: `result = a % b` results in 0

## 7. Built-in Functions for Data Handling

Python provides several built-in functions for data handling. Some useful functions include: - `abs(value)`: Returns the absolute value of a number. - `round(value)`: Rounds a floating-point number to the nearest integer. - `int(x)`: Converts a floating-point number to an integer. - `float(x)`: Converts an integer to a floating-point number.

## 8. Conclusion

In this lecture, we covered the basic data types in Python: integers, floating-point numbers, and strings. We learned how to create and manipulate variables, perform arithmetic operations, and utilize built-in functions for data handling. Understanding these concepts is crucial for writing effective Python programs.

## Quiz Questions

**Q1: What type of number is represented by the value 3.14 in Python?**

- Integer
- Floating-point number
- String
- Boolean

*Correct: Floating-point number*

**Q2: Which operator is used for concatenating strings in Python?**

- +
- -
- \*
- /

*Correct: +*

**Q3: What function would you use to find the length of a string in Python?**

- len()
- length()
- size()
- count()

*Correct: len()*

# Lecture 3: Control Structures: Loops and Conditionals

In this lecture, we will explore control structures in Python, specifically focusing on loops and conditional statements. Control structures are essential for implementing logic in programming, allowing us to control the flow of execution based on certain conditions. By the end of this lecture, you will understand how to use loops to repeat actions and conditionals to execute code based on specific criteria.

## Introduction to Control Structures

Control structures in programming allow us to dictate how our code executes under various circumstances. In Python, the two primary control structures we will discuss today are loops and conditionals.

## Conditionals

Conditionals allow us to execute certain pieces of code based on whether a condition is true or false. The most common conditional statements in Python are `if`, `elif`, and `else`.

### The `if` Statement

The `if` statement executes a block of code if a specified condition is true. The syntax for the `if` statement is as follows:

```
if condition:
 statements
```

#### Example:

```
gpa = 3.4
if gpa > 2.0:
 print("Your application is accepted.")
```

In this example, the message will be printed only if the GPA is greater than 2.0.

### The `if/else` Statement

The `if/else` statement allows us to execute one block of code if the condition is true and another block if it is false. The syntax is:

```
if condition:
 statements
else:
 statements
```

#### Example:

```
gpa = 1.4
if gpa > 2.0:
 print("Welcome to Mars University!")
else:
 print("Your application is denied.")
```

This will print a different message based on the value of `gpa`.

### The `elif` Statement

The `elif` statement allows us to check multiple conditions in sequence. The syntax is:

```
if condition1:
 statements
elif condition2:
 statements
```

```
else:
statements
```

### Example:

```
x = 30
if x <= 15:
y = x + 15
elif x <= 30:
y = x + 30
else:
y = x
print('y =', y)
```

## Loops

Loops are used to execute a block of code repeatedly, either a specific number of times or until a certain condition is met. In Python, we primarily use for loops and while loops.

### The for Loop

The for loop is used to iterate over a sequence (like a list or a range). The syntax is:

```
for variable in iterable:
statements
```

### Example:

```
for x in range(1, 6):
print(x, "squared is", x * x)
```

This will output the squares of the numbers from 1 to 5.

### The range() Function

The range() function generates a sequence of numbers. It can take one, two, or three arguments: - range(stop) - generates numbers from 0 to stop-1. - range(start, stop) - generates numbers from start to stop-1. - range(start, stop, step) - generates numbers from start to stop-1, incrementing by step.

### Example:

```
for x in range(5, 0, -1):
print(x)
print("Blastoff!")
```

This will count down from 5 to 1.

### The while Loop

The while loop repeatedly executes a block of code as long as a specified condition is true. The syntax is:

```
while condition:
statements
```

### Example:

```
number = 1
while number < 200:
print(number)
number = number * 2
```



This will print powers of 2 until the number reaches or exceeds 200.

## Logic and Operators

When working with conditionals and loops, we often use logical and relational operators: - **Relational Operators:** ==, !=, <, >, <=, >= - **Logical Operators:** and, or, not

### Example:

```
if x < 10 and x > 0:
print("x is a positive single-digit number.")
```

## Loop Control Statements

Sometimes, we need to control the flow of loops more precisely. Python provides control statements like break, continue, and pass: - **break:** Exits the closest enclosing loop. - **continue:** Skips the current iteration and jumps to the next iteration of the loop. - **pass:** Does nothing; it is a placeholder statement.

### Example of break and continue:

```
for i in range(10):
 if i == 5:
 break # exits the loop when i is 5
 if i % 2 == 0:
 continue # skips even numbers
 print(i)
```

## Conclusion

In this lecture, we covered the fundamental control structures in Python: loops and conditionals. You learned how to use if, elif, and else statements to implement decision-making in your programs, as well as how to utilize for and while loops to repeat actions. Understanding these control structures is crucial for writing effective and efficient Python code. In the next lecture, we will dive deeper into functions and how they can help organize and modularize our code.

## Quiz Questions

**Q1: What are the primary control structures discussed in the lecture?**

- Functions and Classes
- Loops and Conditionals
- Variables and Data Types
- Modules and Packages

*Correct: Loops and Conditionals*

**Q2: Which statement is used to execute a block of code if a condition is false?**

- if
- elif
- else
- for

*Correct: else*

**Q3: What does the `break` statement do in a loop?**

- Skips the current iteration
- Exits the closest enclosing loop
- Pauses the loop execution
- Continues to the next iteration

*Correct: Exits the closest enclosing loop*